

GOVERNMENT OF KERALA

DEPARTMENT OF TECHNICAL EDUCATION

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

(GOVT. ENGINEERING COLLEGE)

KOTTAYAM - 686501



RECORD BOOK

GOVERNMENT OF KERALA
DEPARTMENT OF TECHNICAL EDUCATION
RAJIV GANDHI INSTITUTE OF TECHNOLOGY
(GOVT. ENGINEERING COLLEGE)
KOTTAYAM - 686501



20MCA241 DATA SCIENCE LAB

Name: VISHNU PRASAD C P

Branch: Master of Computer Applications

Semester: 3

Roll No: 60

CERTIFIED BONAFIDE RECORD WORK DONE BY

Reg No.

STAFF IN CHARGE

INTERNAL EXAMINER

EXTERNAL EXAMINER

Contents

Assignment 1 Review of python programming	1
1.1 Basic data types	2
1.1.1 Numbers	2
1.1.2 Booleans	2
1.1.3 Strings	2
1.2 Containers	3
1.2.1 Lists	3
1.2.2 Slicing	4
1.2.3 Loops	4
1.2.4 List comprehensions	4
1.2.5 Dictionaries	4
1.2.6 Sets	5
1.2.7 Tuples	5
1.3 Functions	5
1.4 Classes	6
1.4.1 inheritance and method overriding	7
1.4.2 class variables and instance variables	7
Assignment 2 Vectorized Computations using Numpy	8
1.1 Matrix	9
1.1.1 Create Matrix	9
1.1.2 Matrix Transpose	9
1.1.3 Matrix with random values	10
1.1.4 Matrix with Zeros	10
1.1.5 Scalar matrix	10
1.1.6 Matrix Computations	11
Assignment 3 Vectorized Computations using TensorFlow	13
1.1 Matrix	14
1.1.1 CREATE MATRIX	14
1.1.2 MATRIX TRANSPOSE	14
1.1.3 MATRIX WITH RANDOM VALUES	15
1.1.4 MATRIX WITH ZEROS	15
1.1.5 SCALAR MATRIX	16
1.1.6 MATRIX COMPUTATIONS	16
Assignment 4 Explore Data and Create Linear Regression Model	18
1.1 Loading the Dataset	20
1.2 Preview of DataFrame	20
1.3 Dimensions of the Dataset	21
1.4 Descriptive Statistics and Interquartile Range Calculation	21
1.5 Data Infomation	23
1.6 Adding and Displaying a New Column	24
1.7 Dropping a Column from the DataFrame	24

1.8	Adding New Instances to the DataFrame	25
1.9	Deleting Specific Rows by Index	26
1.10	Capping House Prices at a Maximum Value	28
1.11	Extracting Locations with Low House Prices	28
1.12	Filling Missing Values with the Average Number of Convenience Stores	30
1.13	Scaling and Normalizing the Distance to the Nearest MRT Station	31
1.14	Plotting the Histogram of House Prices	32
1.15	Creating a Box-and-Whisker Plot of House Prices per Unit Area	32
1.16	Scatterplot of House Price vs. House Age	33
1.17	Scatterplot of House Price vs. Distance to the Nearest MRT Station	34
1.18	Standardizing Features and Preparing Design Matrix	35
1.19	Calculating Regression Parameters Using Matrix Operations	37
1.20	Linear Regression with Manual Gradient Descent and TensorFlow GradientTape	37
1.21	Implementing and Using a Custom Linear Regression Class	40
Assignment 5 Building Machine Learning Models with Scikit-learn		43
1.1	Loading the Dataset	45
1.2	Split the Dataset	45
1.3	Load the Wine Dataset	45
1.4	Preprocess the Data	52
1.5	Apply K-Means Clustering	53
1.6	Print Cluster Labels	54
1.7	Compare Cluster Labels with Actual Labels	54
1.8	Compare Cluster Labels with Actual Labels	55
1.9	Visualize the Clusters	55
1.10	Load the California Housing Dataset	56
1.11	Preprocess the Data	57
1.12	Split the Dataset	57
1.13	Train a Linear Regression Model	57
1.14	Evaluate the Model	57
1.15	Print the Coefficients	58
Assignment 6 Implementing an FCNN from Scratch using TensorFlow		59
1.1	Load and Preprocess Data	62
1.2	Display an Image	62
1.3	Neural Network Parameters	63
1.4	Activation Functions and Encoding	63
1.5	Training Loop	63
1.6	Make Predictions	64
1.7	Evaluate Model	65
1.8	Error, Precision, Recall, f1 score	66

Assignment 1

Review of python programming

Problem Statement

Write Python code to explore and practice with the basic data types, containers, functions, and classes of Python.

1. Start by creating variables of various numeric data types and assigning them values.
2. Print the data types and values of these variables.
3. Perform mathematical operations on these variables.
4. Update the values of these variables.
5. Create boolean variables with True or False values.
6. Print the data types of these boolean variables.
7. Perform Boolean operations on these boolean variables.
8. Create string variables with text values.
9. Print the contents and lengths of these string variables.
10. Concatenate strings.
11. Format strings with variables.
12. Use string methods to manipulate strings by capitalizing, converting to uppercase, justifying, centering, replacing substrings, and stripping whitespace.
13. Create and use Python lists. Perform tasks like appending elements, indexing, slicing, and iterating through the list.
14. Create and use Python tuples. Perform tasks like indexing, slicing, and concatenation.
15. Create and use Python sets. Perform tasks like accessing, adding, deleting set elements.
16. Create and use Python dictionaries. Perform tasks like adding, updating, and removing key-value pairs, and accessing values.
17. Define simple functions with parameters and return values.
18. Call functions with different arguments and use the returned results.
19. Write functions that accept other functions as arguments.

20. Define and use Python classes. Include tasks like creating a class, defining methods, and creating instances.
21. Implement class inheritance and method overriding.
22. Create a class with class variables and instance variables, and demonstrate their usage.

1.1 Basic data types

1.1.1 Numbers

```
1 x = 10
2 print(x)
3 print("Addition",x + 1)
4 print("Subtraction",x - 1)
5 print(" Multiplication",x * 2)
6 print("Exponentiation",x ** 2)
7 print("Division",x / 2)
```

```
10 <class 'int'>
Addition 11
Subtraction 9
Multiplication 20
Exponentiation 100
Division 5
```

1.1.2 Booleans

```
1 t, f = True, False
2 print(type(t))
3 print(t and f) # Logical AND;
4 print(t or f)  # Logical OR;
5 print(not t)   # Logical NOT;
6 print(t != f)  # Logical XOR;
```

```
<class 'bool'>
False True False True
```

1.1.3 Strings

```
1 str1='Hello'
2 str2='World'
3 print(str1, len(str1))
4 str3 = str1 + ' ' + str2
5 print(hw)
6 hw12 = '{} {} {}'.format(str1, str2, 7)
7 print(hw12)
```

```
hello 5
hello world
hello world 7
```

```
1 s = "hello"
2 print(s.capitalize())
3 print(s.upper())
4 print(s.rjust(7))
5 print(s.center(7))
6 print(s.replace('l', '(ell)'))
7 print(' world '.strip())
```

```
Hello
HELLO
hello
    hello
he(ell)(ell)o
world
```

1.2 Containers

1.2.1 Lists

```
1 li = [2, 3, 4, 5]
2 print(li, li[2])
3 print(li[-1])
4 li[2] = 'fig'
5 print(li)
6 li.append('big')
7 print(li)
8 r = li.pop()
9 print(r, li)
```

```
[2, 3, 4, 5] 4
5
[2, 3, 'fig', 5]
[2, 3, 'fig', 5, 'big']
big [2, 3, 'fig', 5]
```

1.2.2 Slicing

```
1 n = list(range(6))
2 print(n)
3 print(n[1:3])
4 print(n[3:])
5 print(n[:3])
6 print(n[:])
7 print(n[:-1])
8 n[2:4] = [8, 9]
9 print(n)
```

```
[0, 1, 2, 3, 4, 5]
[1, 2]
[0, 1, 2]
[3, 4, 5]
[0, 1, 2, 3, 4, 5]
[0, 1, 2, 3, 4]
[0, 1, 8, 9, 4, 5]
```

1.2.3 Loops

```
1 animals = ['cat', 'dog', 'elephant']
2 for animal in animals:
3     print(animal)
```

```
cat
dog
elephant
```

1.2.4 List comprehensions

```
1 num = [1, 2, 3, 4, 5]
2 sq = []
3 for i in num:
4     sq.append(i ** 2)
5 print(sq)
```

```
[1, 4, 9, 16, 25]
```

1.2.5 Dictionaries

```
1 d = {'cat': 'cute', 'dog': 'furry'}
2 print(d['cat'])
3 print('cat' in d)
4 d['fish'] = 'wet'
5 print(d['fish'])
```

```
cute
True
wet
```

1.2.6 Sets

```
1 animals = {'cat', 'dog'}
2 print('cat' in animals)
3 print('fish' in animals)
4 animals.add('cat')
5 print(len(animals))
6 animals.remove('cat')
7 print(len(animals))
```

True

False

3

2

1.2.7 Tuples

```
1 d = {(x, x + 1): x for x in range(10)}
2 t = (5, 6)
3 print(type(t))
4 print(d[t])
5 print(d[(1, 2)])
```

<class 'tuple'>

5

1

1.3 Functions

```
1 def sign(x):
2     if x > 0:
3         return 'positive'
4     elif x < 0:
5         return 'negative'
6     else:
7         return 'zero'
8 for x in [-1, 0, 1]:
9     print(sign(x))
```

negative

zero

positive

```
1 def hello(name, loud=False):
2     if loud:
3         print("HELLO, {}".format(name.upper()))
4     else:
5         print("Hello, {}".format(name))
6 hello("vishnu")
7 hello("Prasad", loud=True)
```

Hello, Vishnu!
HELLO, PRASAD

```
1 def apply_function(func, value):
2     return func(value)
3 def square(x):
4     return x * x
5 def cube(x):
6     return x * x * x
7 print(apply_function(square, 5))
8 print(apply_function(cube, 5))
```

25
125

1.4 Classes

```
1 class Greeter:
2     def __init__(self, name):
3         self.name = name
4     def greet(self, loud=False):
5         if loud:
6             print('HELLO, {}'.format(self.name.upper()))
7         else:
8             print('Hello, {}'.format(self.name))
9 g = Greeter('Fred')
10 g.greet()
11 g.greet(loud=True)
```

Hello, Fred!
HELLO, FRED

1.4.1 inheritance and method overriding

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5 class Dog(Animal):
6     def speak(self):
7         return f"{self.name} barks."
8
9 class Cat(Animal):
10    def speak(self):
11        return f"{self.name} meows."
12
13 print(Dog("Buddy").speak())
14 print(Cat("Whiskers").speak())
```

Buddy barks.

Whiskers meows.

1.4.2 class variables and instance variables

```
1 class MyClass:
2     class_var = "I am a class variable"
3     def __init__(self, instance_var):
4         self.instance_var = instance_var
5 obj = MyClass("I am an instance variable")
6 print(MyClass.class_var)
7 print(obj.class_var)
8 print(obj.instance_var)
```

I am a class variable

I am a class variable

I am an instance variable

Assignment 2

Vectorized Computations using Numpy

Problem Statement

Implement the following computations using NumPy:

1. Create a matrix U of shape (m, n) with input values where m and n are input positive integers.
2. Compute X as the transpose of U .
3. Create a matrix Y of shape $(1, m)$ with random values $\in [0, 1]$.
4. Create a matrix $W1$ of shape (p, n) with random values $\in [0, 1]$ where p is an input positive integer.
5. Create a vector $B1$ of shape $(p, 1)$ with random values $\in [0, 1]$.
6. Create a vector $W2$ of shape $(1, p)$ with all zeros.
7. Create a scalar $B2$ with a random value $\in [0, 1]$.
8. Perform the following computations iteratively 15 times:
 - (a) $Z1 = W1 \cdot X + B1$ (Matrix Multiplication)
 - (b) $A1 = f(Z1)$ where f is a function that returns 0 for negative values and the input value itself otherwise.
 - (c) $Z2 = W2 \cdot A1 + B2$
 - (d) $A2 = g(Z2)$ where g is a function defined as $g(x) = \frac{1}{1+e^{-x}}$.
 - (e) $L = \frac{1}{2}(A2 - Y)^2$
 - (f) $dA2 = A2 - Y$
 - (g) $dZ2 = dA2 \circ gprime(Z2)$ where $gprime(x)$ is a function that returns $g(x) \cdot (1 - g(x))$ and $\circ$ indicates element-wise multiplication.
 - (h) $dA1 = W2^T \cdot dZ2$
 - (i) $dZ1 = dA1 \circ fprime(Z1)$ where $fprime$ is a function that returns 1 for positive values and 0 otherwise and $\circ$ indicates element-wise multiplication.
 - (j) $dW1 = \frac{1}{m} \cdot dZ1 \cdot X^T$
 - (k) $dB1 = \frac{1}{m} \sum dZ1$ (sum along the columns)
 - (l) $dW2 = \frac{1}{m} \cdot dZ2 \cdot A1^T$

(m) $dB2 = \frac{1}{m} \sum dZ2$ (sum along the columns)

(n) Update and print $W1$, $B1$, $W2$, and $B2$ for $\alpha = 0.01$:

i. $W1 = W1 - \alpha \cdot dW1$

ii. $B1 = B1 - \alpha \cdot dB1$

iii. $W2 = W2 - \alpha \cdot dW2$

iv. $B2 = B2 - \alpha \cdot dB2$

1.1 Matrix

1.1.1 Create Matrix

```
1 import numpy as np
2 m=int(input("Enter row size:"))
3 n=int(input("Enter column size:"))
4 print("Enter values in single line(space seperated format):")
5 entries=list(map(int,input().split()))
6 U=np.array(entries).reshape(m,n)
7 print(U)
```

Enter row size: 3

Enter column size: 2

Enter values in single line(space seperated format):

4 5 6 7 8 9

[[4 5]

[6 7]

[8 9]]

1.1.2 Matrix Transpose

```
1 X=U.T
2 print("Transpose of matrix U:")
3 print(X)
```

Transpose of matrix U:

[[4 6 8]

[5 7 9]]

1.1.3 Matrix with random values

```
1 m=5
2 Y=np.random.rand(1,m)
3 print(Y)
4
5 p=int(input("Enter positive value,p :"))
6 n=3
7 W1=np.random.rand(p,n)
8 print("Matrix W1\n",W1)
9
10 p=int(input("Enter a value for p :"))
11 B1=np.random.rand(p,1)
12 print("Vector B1:")
13 print(B1)
```

```
[[0.77443087  0.54251259 0.98966121 0.90284055 0.23850017]]
Enter positive value,p : 2
Matrix W1
[[0.17123873 0.38860149 0.8942191 0.33339399 0.55808285]
 [0.33533201 0.72976372 0.25007668 0.50705875 0.63305019] ]
Enter a value for p : -3
Vector B1:
[[0.65182645]
 [0.54404735]
 [0.85631588]]
```

1.1.4 Matrix with Zeros

```
1 p=int(input("Enter value for p :"))
2 W2=np.zeros((1,p))
3 print(W2)
```

```
Enter value for p : 3
[[0. 0. 0.]]
```

1.1.5 Scalar matrix

```
1 B2=np.random.rand()
2 print("Scalar B2:")
3 print(B2)
```

```
Scalar B2:
0.2917723950656034
```

1.1.6 Matrix Computations

```
1 import numpy as np
2
3 def f(Z):
4     return np.maximum(0,Z)
5
6 def g(Z):
7     return 1/(1+np.exp(-Z))
8
9 def gprime(Z):
10    gz=g(Z)
11    return gz*(1-gz)
12
13 def fprime(Z):
14    return (Z > 0).astype(float)
15
16 alpha=0.01
17 for i in range(15):
18     Z1=np.dot(W1,X) + B1
19
20     A1=f(Z1)
21
22     Z2=np.dot(W2,A1) + B2
23
24     A2=g(Z2)
25
26     L=0.5 * np.square(A2 - Y)
27
28     dA2=A2 -Y
29
30     dZ2=dA2 * gprime(Z2)
31
32     dA1=np.dot(W2.T,dZ2)
33
34     dZ1=dA1 * fprime(Z1)
35
36     dW1=np.dot(dZ1, X.T)
```

```
1     dB1=np.sum(dZ1, axis=1,keepdims=True)/m
2
3     dW2=np.dot(dZ2,A1.T)/m
4
5     dB2=np.sum(dZ2)/m
6
7     W1 -=alpha * dW1
8     B1 -=alpha * dB1
9     W2 -=alpha * dW2
10    B2 -=alpha * dB2
11
12    print("\n Updated W1:\n",W1)
13    print("\n Updated B1:\n",B1)
14    print("\n Updated W2:\n",W2)
15    print("\n Updated B2:\n",B2)
16    print("\n Loss L:\n",L)
```

Updated W1:

```
[[0.91607861 078653667 ]
 [0.7542901  0.6444458333]
 [ 0.31695958 0.90471041]]
```

Updated B1:

```
[[0.27108419]
 [0.56487425]
 [0.45061062]]
```

Updated W2:

```
[[0.00830779  0.00813867 0.00833619]]
```

Updated B2:

```
0.7309443342862254
```

Loss L:

```
[[0.04591595 0.00085737 0.019902 ]]
```

Assignment 3

Vectorized Computations using TensorFlow

Problem Statement

Implement the following computations using TensorFlow:

1. Create a matrix U of shape (m, n) with input values where m and n are input positive integers.
2. Compute X as the transpose of U .
3. Create a matrix Y of shape $(1, m)$ with random values $\in [0, 9]$.
4. Create a matrix $W1$ of shape (p, n) with random values $\in [0, 1]$ where p is an input positive integer.
5. Create a vector $B1$ of shape $(p, 1)$ with random values $\in [0, 1]$.
6. Create a vector $W2$ of shape $(10, p)$ with all zeros.
7. Create a scalar $B2$ with a random value $\in [0, 1]$.
8. Perform the following computations iteratively 15 times:
 - (a) $Z1 = W1 \cdot X + B1$ (Matrix Multiplication)
 - (b) $A1 = \text{ReLU}(Z1)$ where $\text{ReLU}(x)$ is a function that returns 0 for negative values and the input value itself otherwise.
 - (c) $Z2 = W2 \cdot A1 + B2$
 - (d) $A2 = \text{softmax}(Z2)$ where $\text{softmax}(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}$
 - (e) $dZ2 = A2 - \text{one_hot_Y}$ where one_hot_Y is the one-hot encoded form of Y .
 - (f) $dA2 = W2^T \cdot dZ2$
 - (g) $dW2 = \frac{1}{m} \cdot dZ2 \cdot A1^T$
 - (h) $dB2 = \frac{1}{m} \sum dZ2$ (sum along the columns)
 - (i) $dZ1 = dA1 \circ \text{ReLU_deriv}(Z1)$ where $\text{ReLU_deriv}(x)$ returns 1 for positive values and 0 otherwise, and $\circ$ indicates element-wise multiplication.
 - (j) $dA1 = W2^T \cdot dZ1$
 - (k) $dB1 = \frac{1}{m} \sum dZ1$ (sum along the columns)
 - (l) $dW1 = \frac{1}{m} \cdot dZ1 \cdot X^T$
 - (m) Update and print $W1$, $B1$, $W2$, and $B2$ for $\alpha = 0.01$:

- i. $W1 = W1 - \alpha \cdot dW1$
- ii. $B1 = B1 - \alpha \cdot dB1$
- iii. $W2 = W2 - \alpha \cdot dW2$
- iv. $B2 = B2 - \alpha \cdot dB2$

1.1 Matrix

1.1.1 CREATE MATRIX

```

1 import numpy as np
2 import tensorflow as tf
3 def create_matrix(m,n):
4     u=tf.Variable(tf.random.normal(shape=(m,n)))
5     return u
6 m=int(input("enter the number of rows:"))
7 n=int(input("enter the number of columns:"))
8 matrix=create_matrix(m,n)
9 print(matrix.numpy())

```

```

enter the number of rows: 3
enter the number of columns: 4

```

```

[[-1.1837945 -0.33722427  0.23563308  0.16107185]
 [-1.5149251 -0.5944967  1.4439311  1.4481225 ]
 [-0.14096595 -0.60184324  1.3875078  0.17040999]]

```

1.1.2 MATRIX TRANSPOSE

```

1 x=tf.transpose(matrix)
2 print(x.numpy())

```

```

[[-1.1837945 -1.5149251 -0.14096595]
 [-0.33722427 -0.5944967 -0.60184324]
 [ 0.23563308  1.4439311  1.3875078 ]
 [ 0.16107185  1.4481225  0.17040999]]

```

1.1.3 MATRIX WITH RANDOM VALUES

```
1 y=tf.Variable(tf.random.uniform(shape=(1,m),minval=0,maxval=10,dtype=tf.int32)↵
   )
2 print(y.numpy())
3
4 p=int(input("enter the number of rows for w1:"))
5 w1=tf.Variable(tf.random.uniform(shape=(p,n),minval=0,maxval=1,dtype=tf.
6   float32  ))
7 print(w1.numpy())
8
9 B1=tf.Variable(tf.random.uniform(shape=(p,1),minval=0,maxval=1,dtype=tf.
10   float32  ))
11 print(B1.numpy())
```

[[7 3 4]]

enter the number of rows for w1: 3

[[0.41094923 0.64525306 0.21415687 0.05348241]

[0.63344413 0.387007 0.71524847 0.2568928]

[0.39981616 0.39820206 0.48125303 0.20443547]]

[[0.2172625]

[0.86350536]

[0.48577976]]

1.1.4 MATRIX WITH ZEROS

```
1 w2=tf.Variable(tf.zeros(shape=(10,p)))
2 print(w2.numpy())
```

[[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]]

1.1.5 SCALAR MATRIX

```
1 B2=tf.Variable(tf.random.uniform([],minval=-1,maxval=2,dtype=tf.float32))
2 print(B2.numpy())
```

1.9291875

1.1.6 MATRIX COMPUTATIONS

```
1 alpha = 0.01
2
3 def relu(x):
4     return tf.maximum(0, x)
5
6 def relu_deriv(x):
7     return tf.where(x > 0, tf.ones_like(x), tf.zeros_like(x))
8
9 def softmax(x):
10    return tf.nn.softmax(x, axis=0)
11
12 def one_hot_encode(y, depth):
13    return tf.one_hot(y, depth)
14
15 for i in range(15):
16    Z1 = tf.matmul(w1, x) + B1
17    A1 = relu(Z1)
18    Z2 = tf.matmul(w2, A1) + B2
19    A2 = softmax(Z2)
20
21    y_one_hot = one_hot_encode(y, 10)
22    y_one_hot = tf.transpose(y_one_hot, perm=[2, 1, 0])
23    y_one_hot = tf.reshape(y_one_hot, [10, 3])
24
25    L = 0.5 * tf.reduce_sum(tf.square(A2 - tf.cast(y_one_hot, tf.float32)))
26
27    dZ2 = A2 - tf.cast(y_one_hot, dtype=tf.float32)
28    dW2 = (1 / m) * tf.matmul(dZ2, A1, transpose_b=True)
29    dB2 = (1 / m) * tf.reduce_sum(dZ2, axis=1, keepdims=True)
30    dA1 = tf.matmul(w2, dZ2, transpose_a=True)
31    dZ1 = dA1 * relu_deriv(Z1)
32    dW1 = (1 / m) * tf.matmul(dZ1, x, transpose_b=True)
33    dB1 = (1 / m) * tf.reduce_sum(dZ1, axis=1, keepdims=True)
34
35    w1.assign_sub(alpha * dW1)
36    B1.assign_sub(alpha * dB1)
37    w2.assign_sub(alpha * dW2)
38    B2.assign_sub(alpha * dB2)
39
40    print(f"Updated W1:\n{w1.numpy()}")
```

```
41     print(f"Updated B1:\n{B1.numpy()}")
42     print(f"Updated W2:\n{w2.numpy()}")
43     print(f"Updated B2:\n{B2.numpy()}")
```

Updated W1:

```
[[0.41094914 0.6452527 0.21415766 0.05348251]
 [0.6334481 0.3870048 0.71525824 0.2568947 ]
 [0.39982006 0.39820063 0.4812594 0.20443718]]
```

Updated B1:

```
[[0.21726307]
 [0.86350536]
 [0.48577976]]
```

Updated W2:

```
[[ -1.5348143e-11 -5.6607086e-10 -3.0818986e-10]
 [ -1.5348143e-11 -5.6607086e-10 -3.0818986e-10]
 [ -1.5348143e-11 -5.6607086e-10 -3.0818986e-10]
 [ -1.7174566e-04  8.5782167e-04  7.8074128e-04]
 [  3.4333178e-04  4.1825897e-03  2.5001424e-03]
 [ -1.5348143e-11 -5.6607086e-10 -3.0818986e-10]
 [ -1.5348143e-11 -5.6607086e-10 -3.0818986e-10]
 [ -1.7152089e-04 -5.0434656e-03 -3.2830592e-03]
 [  1.5348143e-11  5.6607086e-10  3.0818992e-10]
 [  1.5348143e-11  5.6607086e-10  3.0818992e-10]]
```

Updated B2:

```
1.929187536239624
```

Assignment 4

Explore Data and Create Linear Regression Model

Problem Statement

Implement the following computations using Pandas and TensorFlow:

1. Load the dataset and import it into a Pandas DataFrame.
2. Display the first five rows and the last three rows of the dataset.
3. Get the dimensions (number of rows and columns) of the dataset.
4. Generate descriptive statistics (mean, median, standard deviation, five-point summary, IQR, etc.) for the data.
5. Print a concise summary of the dataset as information on data types (schema) and missing values.
6. Add a new column named “X22” by converting the “house age” from years to days.
7. Delete the column “X22” from the dataset.
8. Create three new instances synthetically and add them to the dataset.
9. Delete the newly inserted three instances from the dataset.
10. Update the “house price of unit area” to 110, provided it is currently greater than the amount.
11. Find the latitude and longitude of the houses whose prices are less than or equal to 20.
12. Add the missing convenience store values of instances by calculating the average number of convenience stores.
13. Find the normalized distance to the nearest train station by performing:
 - (a) Z-score normalization.
 - (b) Min-max normalization.
 - (c) Decimal scaling.
14. Generate the following basic visualizations using Seaborn. Customize your visualizations by adding titles, labels, legends, and appropriate color schemes.
 - (a) Create a histogram for the “Y house price of unit area” attribute.

- (b) Create a box-and-whisker plot for the ‘Y house price of unit area’ attribute.
 - (c) Create a scatter plot showing house prices against house age.
 - (d) Add a second scatter plot showing house prices against distance to the nearest MRT station.
15. Form the Design Matrix X of shape $m \times n + 1$ in order to apply normal equation method where m is the number of training examples and n is the number of input features. Only use the two normalized input features ‘X2 house age’ and ‘X3 distance to the nearest MRT station’ from the dataset as second and third columns respectively and all 1 s as the first column. Also, form output vector Y of shape $m \times 1$.
 16. Find the parameter vector W using the normal equation method as $W = (X^T X)^{-1} X^T Y$.
 17. Implement the gradient descent algorithm with the following steps:
 - Form the Design Matrix X of shape $n \times m$. Only use the two normalized input features *X2 house age* and *X3 distance to the nearest MRT station* and the output vector Y of shape $1 \times m$.
 - Initialize the parameter vector W of shape $1 \times n$ and bias b (scalar).
 - Repeat the following steps for a certain number of iterations with learning rate $\alpha = 0.01$, and print the final parameter values:
 - (a) Calculate the prediction $\hat{Y} = WX + b$.
 - (b) Compute the loss $L = \frac{1}{2} \times (\hat{Y} - Y)^2$.
 - (c) Compute the error $E = \hat{Y} - Y$.
 - (d) Compute the gradient with respect to W as $dW = \frac{1}{m} E \cdot X^T$ and with respect to b as $db = \frac{1}{m} \times E$ (sum over the columns).
 - (e) Update $W = W - \alpha dW$ and $b = b - \alpha db$.
 - Use TensorFlow’s GradientTape() to automatically calculate the gradients in step (d) and redo the training steps to print the final parameter values.
 18. Define a class to create a Linear Regression model with methods fit and predict. Use the above iterative process to implement the model’s training within the fit method.

1.1 Loading the Dataset

```
1 import pandas as pd
2 csv_url = 'https://docs.google.com/spreadsheets/d/1HgdAx_pUi8gzdBnk203EZ1h-90↵
           cmhdLTpHeWwmXpQo/export?format=csv'
3 df = pd.read_csv(csv_url)
```

1.2 Preview of DataFrame

```
1 print(df.head())
2 print(df.tail(3))
```

```
X1 transaction date X2 house age X3 distance to the nearest MRT station \
0 2012.917 32.0 84.87882
1 2012.917 19.5 306.59470
2 2013.583 13.3 561.98450
3 2013.500 13.3 561.98450
4 2012.833 5.0 390.56840
```

```
X4 number of convenience stores X5 latitude X6 longitude \
0 10.0 24.98298 121.54024
1 9.0 24.98034 121.53951
2 5.0 24.98746 121.54391
3 5.0 24.98746 121.54391
4 5.0 24.97937 121.54245
```

```
Y house price of unit area
0 37.9
1 42.2
2 47.3
3 54.8
4 43.1
```

```
X1 transaction date X2 house age \
412 2013.000 8.1
413 2013.500 6.5
414 2013.167 1.9
```

```
X3 distance to the nearest MRT station X4 number of convenience stores \
412 104.81010 5.0
413 90.45606 9.0
414 355.00000 NaN
```

```
X5 latitude X6 longitude Y house price of unit area
412 24.96674 121.54067 52.5
413 24.97433 121.54310 63.9
414 24.97293 121.54026 40.5
```

1.3 Dimensions of the Dataset

```
1 dimensions=df.shape
2 print(f"\n Dimensions of the dataset:{dimensions[0]} rows and {dimensions[1]} ←
   columns")
```

Dimensions of the dataset:415 rows and 7columns

1.4 Descriptive Statistics and Interquartile Range Calculation

```
1 print("Descriptive Statistics::")
2 print(df.describe())
3 print("\nMedian:",df.median())
4 Q1=df.quantile(0.25)
5 Q3=df.quantile(0.75)
6 IQR=Q3-Q1
7 print("Interquartille Range(IQR):",IQR)
```

Descriptive Statistics::

```
X1 transaction date X2 house age \
count 415.000000 415.000000
mean 2013.149014 17.674458
std 0.281628 11.405161
min 2012.667000 0.000000
25% 2012.917000 8.950000
50% 2013.167000 16.100000
75% 2013.417000 28.100000
max 2013.583000 43.800000
```

```
X3 distance to the nearest MRT station \
count 415.000000
mean 1082.129338
std 1261.092057
```

min 23.382840
25% 289.324800
50% 492.231300
75% 1452.760000
max 6488.021000

X4 number of convenience stores X5 latitude X6 longitude \
count 414.000000 415.000000 415.000000
mean 4.094203 24.969039 121.533378
std 2.945562 0.012397 0.015332

min 0.000000 24.932070 121.473530
25% 1.000000 24.963010 121.528570
50% 4.000000 24.971100 121.538630
75% 6.000000 24.977450 121.543300
max 10.000000 25.014590 121.566270

Y house price of unit area

count 415.000000
mean 37.986265
std 13.590608
min 7.600000
25% 27.700000
50% 38.500000
75% 46.600000
max 117.500000

Median: X1 transaction date 2013.16700

X2 house age 16.10000

X3 distance to the nearest MRT station 492.23130

X4 number of convenience stores 4.00000

X5 latitude 24.97110

X6 longitude 121.53863

Y house price of unit area 38.50000

dtype: float64

Interquartille Range(IQR): X1 transaction date 0.50000

X2 house age 19.15000

X3 distance to the nearest MRT station 1163.43520

X4 number of convenience stores 5.00000
X5 latitude 0.01444
X6 longitude 0.01473
Y house price of unit area 18.90000
dtype: float64

1.5 Data Infromation

```
1 print("Dataset Infromation")
2 print(df.info())
3 print("Missing Values:",df.isnull().sum())
```

```
Dataset Infromation
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 415 entries, 0 to 414
Data columns (total 7 columns):
# Column Non-Null Count Dtype
---  ---
0 X1 transaction date 415 non-null float64
1 X2 house age 415 non-null float64
2 X3 distance to the nearest MRT station 415 non-null float64
3 X4 number of convenience stores 414 non-null float64
4 X5 latitude 415 non-null float64
5 X6 longitude 415 non-null float64
6 Y house price of unit area 415 non-null float64
dtypes: float64(7)
memory usage: 22.8 KB
None
Missing Values: X1 transaction date 0
X2 house age 0
X3 distance to the nearest MRT station 0
X4 number of convenience stores 1
X5 latitude 0
X6 longitude 0
Y house price of unit area 0
dtype: int64
```

1.6 Adding and Displaying a New Column

```
1 df['X22']=df['X2 house age']*365
2 print(df.head())
```

```
X1 transaction date X2 house age X3 distance to the nearest MRT station \
0 2012.917 32.0 84.87882
1 2012.917 19.5 306.59470
2 2013.583 13.3 561.98450
3 2013.500 13.3 561.98450
4 2012.833 5.0 390.56840
```

```
X4 number of convenience stores X5 latitude X6 longitude \
0 10.0 24.98298 121.54024
1 9.0 24.98034 121.53951
2 5.0 24.98746 121.54391
3 5.0 24.98746 121.54391
4 5.0 24.97937 121.54245
```

```
Y house price of unit area X22
0 37.9 11680.0
1 42.2 7117.5
2 47.3 4854.5
3 54.8 4854.5
4 43.1 1825.0
```

1.7 Dropping a Column from the DataFrame

```
1 df.drop('X22', axis=1, inplace=True)
2 print(df.head())
```

```
X1 transaction date X2 house age X3 distance to the nearest MRT station \
0 2012.917 32.0 84.87882
1 2012.917 19.5 306.59470
2 2013.583 13.3 561.98450
3 2013.500 13.3 561.98450
4 2012.833 5.0 390.56840
```

```

X4 number of convenience stores X5 latitude X6 longitude \
0 10.0 24.98298 121.54024
1 9.0 24.98034 121.53951
2 5.0 24.98746 121.54391
3 5.0 24.98746 121.54391
4 5.0 24.97937 121.54245

Y house price of unit area
0 37.9
1 42.2
2 47.3
3 54.8
4 43.1

```

1.8 Adding New Instances to the DataFrame

```

1 new_instances = pd.DataFrame({
2     'house_age': [5, 10, 15],
3     'distance_to_MRT': [300, 450, 600],
4     'house_price': [25, 30, 35],
5 })
6 df = pd.concat([df, new_instances], ignore_index=True)
7 print(df.tail(5))

```

```

X1 transaction date X2 house age \
413 2013.500 6.5
414 2013.167 1.9
415 NaN NaN
416 NaN NaN
417 NaN NaN

X3 distance to the nearest MRT station X4 number of convenience stores \
413 90.45606 9.0
414 355.00000 NaN
415 NaN NaN
416 NaN NaN
417 NaN NaN

X5 latitude X6 longitude Y house price of unit area house_age \

```

```

413 24.97433 121.54310 63.9 NaN
414 24.97293 121.54026 40.5 NaN
415 NaN NaN NaN 5.0
416 NaN NaN NaN 10.0
417 NaN NaN NaN 15.0

```

```

distance_to_MRT house_price
413 NaN NaN
414 NaN NaN
415 300.0 25.0
416 450.0 30.0
417 600.0 35.0

```

1.9 Deleting Specific Rows by Index

```

1 indices_to_delete=df.index[-3:]
2 df=df.loc[~df.index.isin(indices_to_delete)]
3 print(df)

```

```

X1 transaction date X2 house age \
0 2012.917 32.0
1 2012.917 19.5
2 2013.583 13.3
3 2013.500 13.3
4 2012.833 5.0
... ..
410 2012.667 5.6
411 2013.250 18.8
412 2013.000 8.1
413 2013.500 6.5
414 2013.167 1.9

```

```

X3 distance to the nearest MRT station X4 number of convenience stores \
0 84.87882 10.0
1 306.59470 9.0
2 561.98450 5.0
3 561.98450 5.0
4 390.56840 5.0

```

```

.. .. .
410 90.45606 9.0
411 390.96960 7.0
412 104.81010 5.0
413 90.45606 9.0
414 355.00000 NaN

X5 latitude X6 longitude Y house price of unit area house_age \
0 24.98298 121.54024 37.9 NaN
1 24.98034 121.53951 42.2 NaN
2 24.98746 121.54391 47.3 NaN
3 24.98746 121.54391 54.8 NaN
4 24.97937 121.54245 43.1 NaN
.. .. .
410 24.97433 121.54310 50.0 NaN
411 24.97923 121.53986 40.6 NaN
412 24.96674 121.54067 52.5 NaN
413 24.97433 121.54310 63.9 NaN
414 24.97293 121.54026 40.5 NaN

distance_to_MRT house_price
0 NaN NaN
1 NaN NaN
2 NaN NaN
3 NaN NaN
4 NaN NaN
.. .. .
410 NaN NaN
411 NaN NaN
412 NaN NaN
413 NaN NaN
414 NaN NaN
[415 rows x 10 columns]

```

1.10 Capping House Prices at a Maximum Value

```
1 df.loc[df['Y house price of unit area'] > 110, 'Y house price of unit area'] ←  
    =110  
2 print(df.head())
```

```
X1 transaction date X2 house age X3 distance to the nearest MRT station \  
0 2012.917 32.0 84.87882  
1 2012.917 19.5 306.59470  
2 2013.583 13.3 561.98450  
3 2013.500 13.3 561.98450  
4 2012.833 5.0 390.56840
```

```
X4 number of convenience stores X5 latitude X6 longitude \  
0 10.0 24.98298 121.54024  
1 9.0 24.98034 121.53951  
2 5.0 24.98746 121.54391  
3 5.0 24.98746 121.54391  
4 5.0 24.97937 121.54245
```

```
Y house price of unit area house_age distance_to_MRT house_price  
0 37.9 NaN NaN NaN  
1 42.2 NaN NaN NaN  
2 47.3 NaN NaN NaN  
3 54.8 NaN NaN NaN  
4 43.1 NaN NaN NaN
```

1.11 Extracting Locations with Low House Prices

```
1 location=df.loc[df['Y house price of unit area']<=20,['X5 latitude','X6  
2 longitude ']]  
3 print(location)
```

```
X5 latitude X6 longitude  
8 24.95095 121.48458  
40 24.94155 121.50381  
41 24.94297 121.50342  
48 24.94684 121.49578
```

49 24.94925 121.49542
55 24.94968 121.53009
73 24.94155 121.50381
83 24.96056 121.50831
87 24.94297 121.50342
93 24.94920 121.53076
113 24.96172 121.53812
116 24.94375 121.47883
117 24.93885 121.50383
155 24.94155 121.50381
156 24.94883 121.52954
162 24.94297 121.50342
170 24.94741 121.49628
176 24.94867 121.49507
180 24.94898 121.49621
183 24.94155 121.50381
226 24.94155 121.50381
229 24.94890 121.53095
231 24.94235 121.50357
232 24.95032 121.49587
249 24.95743 121.47516
251 24.94960 121.53018
255 24.95095 121.48458
298 24.94155 121.50381
309 24.94883 121.52954
320 24.93885 121.50383
329 24.93885 121.50383
330 24.94935 121.53046
331 24.94826 121.49587
347 24.95719 121.47353
384 24.94297 121.50342
409 24.94155 121.50381

1.12 Filling Missing Values with the Average Number of Convenience Stores

```
1 avg_stores = df['X4 number of convenience stores'].mean()
2 df['X4 number of convenience stores'] = df['X4 number of convenience stores'].
3   fillna (avg_stores)
4 print(df.head())
```

```
X1 transaction date X2 house age X3 distance to the nearest MRT station \
0 2012.917 32.0 84.87882
1 2012.917 19.5 306.59470
2 2013.583 13.3 561.98450
3 2013.500 13.3 561.98450
4 2012.833 5.0 390.56840
```

```
X4 number of convenience stores X5 latitude X6 longitude \
0 10.0 24.98298 121.54024
1 9.0 24.98034 121.53951
2 5.0 24.98746 121.54391
3 5.0 24.98746 121.54391
4 5.0 24.97937 121.54245
```

```
Y house price of unit area house_age distance_to_MRT house_price
0 37.9 NaN NaN NaN
1 42.2 NaN NaN NaN
2 47.3 NaN NaN NaN
3 54.8 NaN NaN NaN
4 43.1 NaN NaN NaN
```

1.13 Scaling and Normalizing the Distance to the Nearest MRT Station

```
1 df['distance_zscore'] = (df['X3 distance to the nearest MRT station'] - df['↵
    X 3
2     distance    to the nearest MRT station'].mean()) / df['X3 distance to the
3     nearest    MRT station'].std()
4 min_dist=df['X3 distance to the nearest MRT station'].min()
5 max_dist=df['X3 distance to the nearest MRT station'].max()
6 df['distance_minmax']=(df['X3 distance to the nearest MRT station']-min_dist)/
7     (max_dist-min_dist)
8 max_abs_dist=df['X3 distance to the nearest MRT station'].abs().max()
9 scaling_factor = 10 ** len(str(int(max_abs_dist)))
10 df['distance_decimal_sacling']=df['X3 distance to the nearest MRT station']/
11     scaling_factor
12 print(df.head())
```

```
X1 transaction date X2 house age X3 distance to the nearest MRT station \
0 2012.917 32.0 84.87882
1 2012.917 19.5 306.59470
2 2013.583 13.3 561.98450
3 2013.500 13.3 561.98450
4 2012.833 5.0 390.56840
```

```
X4 number of convenience stores X5 latitude X6 longitude \
0 10.0 24.98298 121.54024
1 9.0 24.98034 121.53951
2 5.0 24.98746 121.54391
3 5.0 24.98746 121.54391
4 5.0 24.97937 121.54245
```

```
Y house price of unit area house_age distance_to_MRT house_price \
0 37.9 NaN NaN NaN
1 42.2 NaN NaN NaN
2 47.3 NaN NaN NaN
3 54.8 NaN NaN NaN
4 43.1 NaN NaN NaN
```

```
distance_zscore distance_minmax distance_decimal_sacling
0 -0.790783 0.009513 0.008488
1 -0.614971 0.043809 0.030659
2 -0.412456 0.083315 0.056198
```

3 -0.412456 0.083315 0.056198
4 -0.548383 0.056799 0.039057

1.14 Plotting the Histogram of House Prices

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 sns.histplot(df['Y house price of unit area'],kde=True)
4 plt.title("Histogram for house price of unit area")
5 plt.xlabel("House price")
6 plt.ylabel("Frequency")
7 plt.show()
```

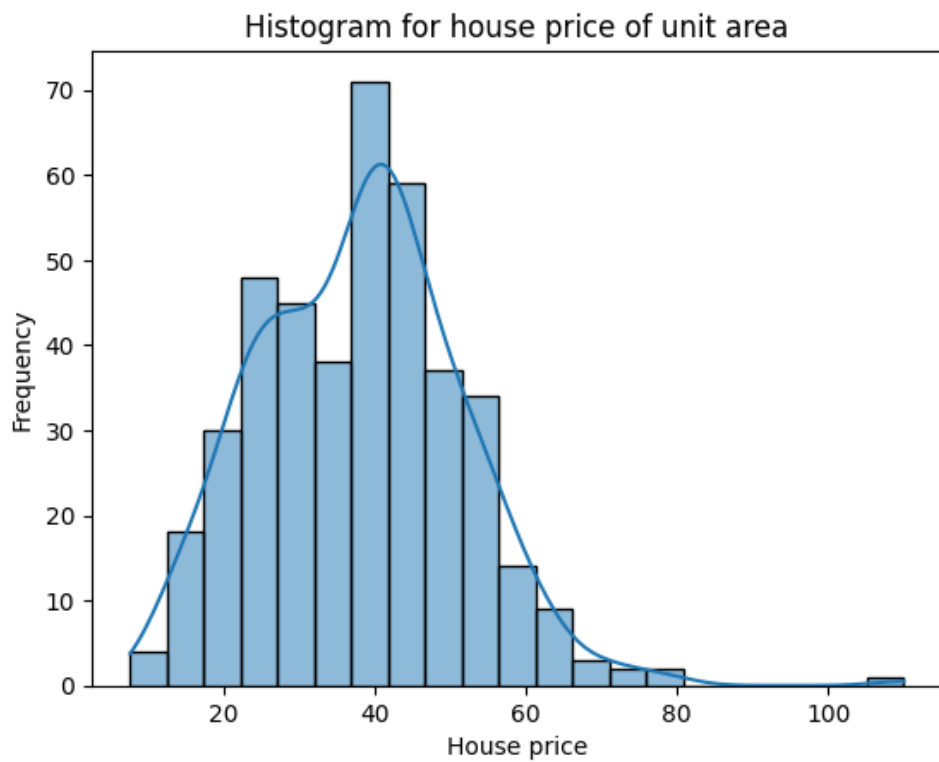


Figure 1.1: Histogram of house price of unit area

1.15 Creating a Box-and-Whisker Plot of House Prices per Unit Area

```

1 sns.boxplot(y=df['Y house price of unit area'])
2 plt.title("Box plot for house price of unit area")
3 plt.ylabel('Y house price of unit area')
4 plt.show()

```

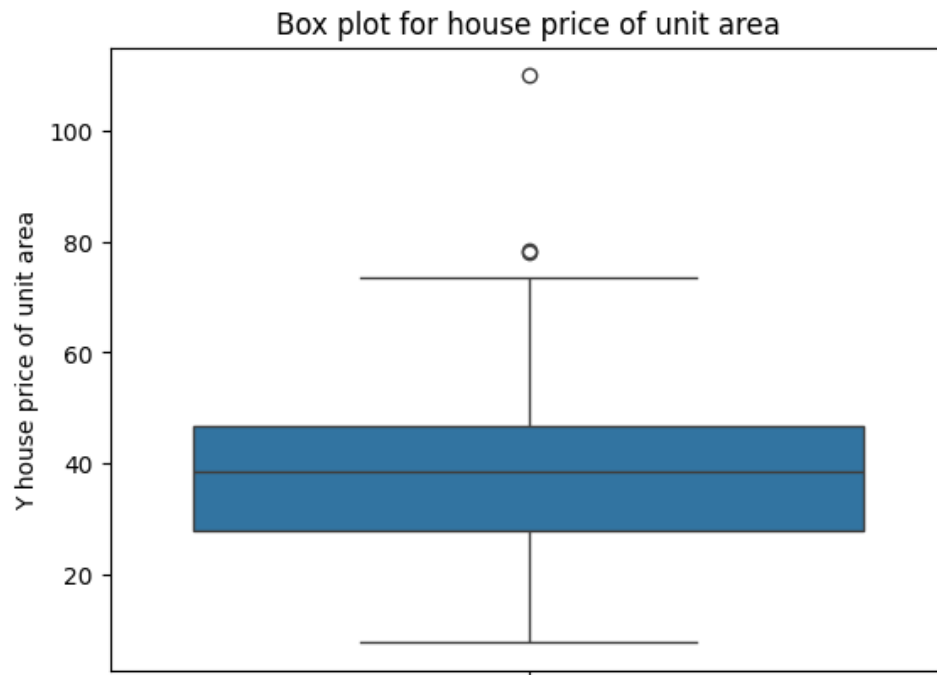


Figure 1.2: Histogram of house price of unit area

1.16 Scatterplot of House Price vs. House Age

```

1 sns.scatterplot(x='X2 house age',y='Y house price of unit area',data=df)
2 plt.title("House price Vs House age")
3 plt.xlabel('X2 house age')
4 plt.ylabel('Y house price of unit area')
5 plt.show()

```

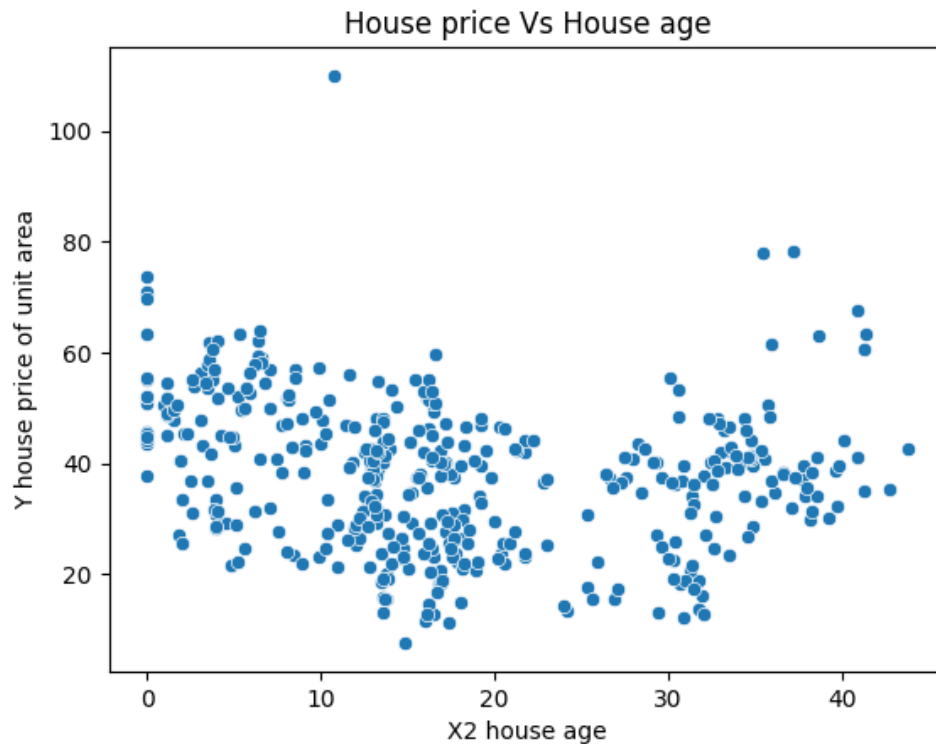


Figure 1.3: Histogram of house price of unit area

1.17 Scatterplot of House Price vs. Distance to the Nearest MRT Station

```

1 plt.figure(figsize=(10,6))
2 sns.scatterplot(x=df['X3 distance to the nearest MRT station'], y=df['Y ↔
  house
3 price of unit area'], color='purple')
4 plt.xlabel('distance to the nearst MRT station')
5 plt.ylabel('House Price of Unit Area')
6 plt.show()

```

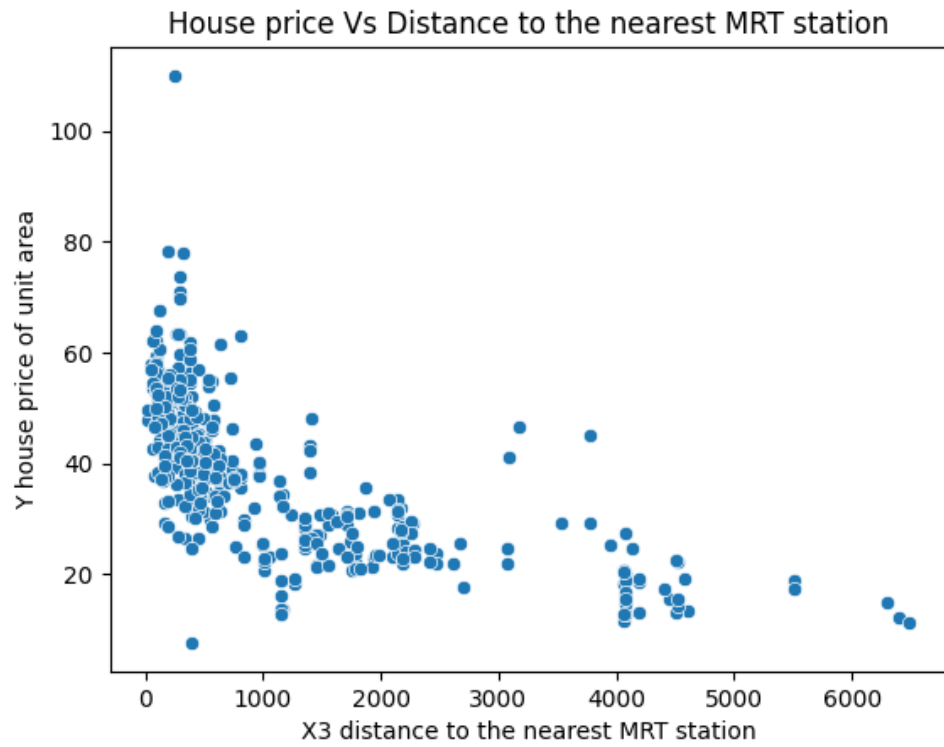


Figure 1.4: Histogram of house price of unit area

1.18 Standardizing Features and Preparing Design Matrix

```

1 import numpy as np
2 df['X2'] = (df['X2 house age'] - df['X2 house age'].mean()) / df['X2 house
3 age'].std()
4 df['X3'] = (df['X3 distance to the nearest MRT station'] - df['X3 distance to
5 the nearest MRT station'].mean()) / df['X3 distance to the nearest MRT
6 station'].std()
7 X = np.c_[np.ones(df.shape[0]), df[['X2 house age', 'X3 distance to the
8 MRT station']]]
9 Y = df['Y house price of unit area'].values
10 print('matrix X:',X)
11 print('output Vector Y:',Y)

```

```

matrix X: [[ 1.  1.25605784 -0.79078328]
[ 1.  0.16006281 -0.61497068]
[ 1. -0.38355072 -0.41245588]
...
[ 1. -0.83948465 -0.77497851]
[ 1. -0.97977201 -0.78636074]
[ 1. -1.38309818 -0.57658704]]

output Vector Y: [ 37.9 42.2 47.3 54.8 43.1 32.1 40.3 46.7 18.8 22.1
41.4 58.1
39.3 23.8 34.3 50.5 70.1 37.4 42.3 47.7 29.3 51.6 24.6 47.9
38.8 27. 56.2 33.6 47. 57.1 22.1 25. 34.2 49.3 55.1 27.3
22.9 25.3 47.7 46.2 15.9 18.2 34.7 34.1 53.9 38.3 42. 61.5
13.4 13.2 44.2 20.7 27. 38.9 51.7 13.7 41.9 53.5 22.6 42.4
21.3 63.2 27.7 55. 25.3 44.3 50.7 56.8 36.2 42. 59. 40.8
36.3 20. 54.4 29.5 36.8 25.6 29.8 26.5 40.3 36.8 48.1 17.7
43.7 50.8 27. 18.3 48. 25.3 45.4 43.2 21.8 16.1 41. 51.8
59.5 34.6 51. 62.2 38.2 32.9 54.4 45.7 30.5 71. 47.1 26.6
34.1 28.4 51.6 39.4 23.1 7.6 53.3 46.4 12.2 13. 30.6 59.6
31.3 48. 32.5 45.5 57.4 48.6 62.9 55. 60.7 41. 37.5 30.7
37.5 39.5 42.2 20.8 46.8 47.4 43.5 42.5 51.4 28.9 37.5 40.1
28.4 45.5 52.2 43.2 45.1 39.7 48.5 44.7 28.9 40.9 20.7 15.6
18.3 35.6 39.4 37.4 57.8 39.6 11.6 55.5 55.2 30.6 73.6 43.4
37.4 23.5 14.4 58.8 58.1 35.1 45.2 36.5 19.2 42. 36.7 42.6
15.5 55.9 23.6 18.8 21.8 21.5 25.7 22. 44.3 20.5 42.3 37.8

```

```

42.7 49.3 29.3 34.6 36.6 48.2 39.1 31.6 25.5 45.9 31.5 46.1
26.6 21.4 44. 34.2 26.2 40.9 52.2 43.5 31.1 58. 20.9 48.1
39.7 40.8 43.8 40.2 78.3 38.5 48.5 42.3 46. 49. 12.8 40.2
46.6 19. 33.4 14.7 17.4 32.4 23.9 39.3 61.9 39. 40.6 29.7
28.8 41.4 33.4 48.2 21.7 40.8 40.6 23.1 22.3 15. 30. 13.8
52.7 25.9 51.8 17.4 26.5 43.9 63.3 28.8 30.7 24.4 53. 31.7
40.6 38.1 23.7 41.1 40.1 23. 110. 26.5 40.5 29.3 41. 49.7
34. 27.7 44. 31.1 45.4 44.8 25.6 23.5 34.4 55.3 56.3 32.9
51. 44.5 37. 54.4 24.5 42.5 38.1 21.8 34.1 28.5 16.7 46.1
36.9 35.7 23.2 38.4 29.4 55. 50.2 24.7 53. 19.1 24.7 42.2
78. 42.8 41.6 27.3 42. 37.5 49.8 26.9 18.6 37.7 33.1 42.5
31.3 38.1 62.1 36.7 23.6 19.2 12.8 15.6 39.6 38.4 22.8 36.5
35.6 30.9 36.3 50.4 42.9 37. 53.5 46.6 41.2 37.9 30.8 11.2
53.7 47. 42.3 28.6 25.7 31.3 30.1 60.7 45.3 44.9 45.1 24.7
47.1 63.3 40. 48. 33.1 29.5 24.8 20.9 43.1 22.8 42.1 51.7
41.5 52.2 49.5 23.8 30.5 56.8 37.4 69.7 53.3 47.3 29.3 40.3
12.9 46.6 55.3 25.6 27.3 67.7 38.6 31.3 35.3 40.3 24.7 42.5
31.9 32.2 23. 37.3 35.5 27.7 28.5 39.7 41.2 37.2 40.5 22.3
28.1 15.4 50. 40.6 52.5 63.9 40.5]

```

1.19 Calculating Regression Parameters Using Matrix Operations

```

1 X_transpose = X.T
2 X_transpose_X = X_transpose @ X
3 X_transpose_Y = X_transpose @ Y
4 X_transpose_X_inv = np.linalg.inv(X_transpose_X)
5 W = X_transpose_X_inv @ X_transpose_Y
6 print(W)

```

```
[37.96819277 -2.60318439 -9.07079233]
```

1.20 Linear Regression with Manual Gradient Descent and TensorFlow GradientTape

```

1 import numpy as np
2 import pandas as pd
3 import tensorflow as tf
4 df['X2 house age'] = (df['X2 house age'] - df['X2 house age'].mean()) / df['↵
    X 2
5     house    age'].std()
6 df['X3 distance to the nearest MRT station'] = (df['X3 distance to the ↵
    nearest
7     MRT    station'] - df['X3 distance to the nearest MRT station'].mean()) /
8     df ['X3 distance to the nearest MRT station'].std()
9 X = df[['X2 house age', 'X3 distance to the nearest MRT station']].values
10 Y = df['Y house price of unit area'].values
11 m, n = X.shape
12 X = np.hstack((np.ones((m, 1)), X))
13 W = np.zeros(n + 1)
14 alpha = 0.01
15 iterations = 1000
16 for i in range(iterations):
17     Y_hat = X @ W
18     error = Y_hat - Y
19     dW = (1 / m) * (X.T @ error)
20     W = W - alpha * dW
21 print("Final parameter values using manual gradient descent:")
22 print(f"Weights: {W[1:]}")
23 print(f"Bias: {W[0]}")
24 X_tf = tf.convert_to_tensor(X, dtype=tf.float64)
25 Y_tf = tf.convert_to_tensor(Y, dtype=tf.float64)
26 W_tf = tf.Variable(tf.zeros(n + 1, dtype=tf.float64))
27 def predict(X, W):
28     return tf.linalg.matvec(X, W)
29 def loss(Y_true, Y_pred):
30     return tf.reduce_mean(tf.square(Y_pred - Y_true)) / 2
31 alpha = 0.01
32 iterations = 1000
33 for i in range(iterations):
34     with tf.GradientTape() as tape:
35         Y_hat_tf = predict(X_tf, W_tf)
36         current_loss = loss(Y_tf, Y_hat_tf)
37         gradients = tape.gradient(current_loss, [W_tf])
38         W_tf.assign_sub(alpha * gradients[0])
39 print("\nFinal parameter values using TensorFlow GradientTape:")
40 print(f"Weights: {W_tf.numpy()[1:]}")
41 print(f"Bias: {W_tf.numpy()[0]}")

```

Final parameter values using manual gradient descent:
Weights: [-2.60317734 -9.07040791]
Bias: 37.966553636840494

Final parameter values using TensorFlow GradientTape:
Weights: [-2.60317734 -9.07040791]
Bias: 37.966553636840494

1.21 Implementing and Using a Custom Linear Regression Class

```
1 import numpy as np
2 import pandas as pd
3 class LinearRegression:
4     def _init_(self, learning_rate=0.01, iterations=1000):
5         self.learning_rate = learning_rate
6         self.iterations = iterations
7         self.W = None
8         self.b = 0
9     def fit(self, X, Y):
10         m, n = X.shape
11         self.W = np.zeros(n + 1)
12         X = np.hstack((np.ones((m, 1)), X))
13         for _ in range(self.iterations):
14             Y_hat = X @ self.W
15             error = Y_hat - Y
16             dW = (1 / m) * (X.T @ error)
17             self.W -= self.learning_rate * dW
18     def predict(self, X):
19         m, n = X.shape
20         X = np.hstack((np.ones((m, 1)), X))
21         return X @ self.W
22 df['X2 house age'] = (df['X2 house age'] - df['X2 house age'].mean()) / df['↵
23         X 2
24         house age'].std()
25 df['X3 distance to the nearest MRT station'] = (df['X3 distance to the ↵
26         nearest
27         MRT station'] - df['X3 distance to the nearest MRT station'].mean()) /
28         df ['X3 distance to the nearest MRT station'].std()
29 X = df[['X2 house age', 'X3 distance to the nearest MRT station']].values
30 Y = df['Y house price of unit area'].values
31 model = LinearRegression(learning_rate=0.01, iterations=1000)
32 model.fit(X, Y)
33 predictions = model.predict(X)
34 print(f"Predictions: {predictions}")
35 print(f"Weights: {model.W[1:]}")
36 print(f"Bias: {model.W[0]}")
```

```
Predictions: [41.86953929 43.12791666 42.70614729 42.70614729 45.83349613
32.51943575
37.42508464 43.08192167 2.90315638 32.87276592 38.92644444 47.6953357
43.27632103 27.36475073 38.39294647 37.46956467 47.67650069 43.22044151
43.27871698 49.27333867 32.38753188 45.37936017 36.64587519 45.47065839
37.28794476 32.39479894 46.3153956 45.42177422 41.39191154 44.91777249
11.36444617 37.49388379 37.61932124 43.68994707 44.79180393 17.27005032
```

32.51112121 37.26213818 44.91933732 44.00534079 17.31984506 16.70042522
37.80799402 38.24401854 45.33059618 37.91426679 41.49390957 36.98135512
11.13348403 10.63269998 41.14445176 29.99246257 32.11022266 43.20784736
44.02816535 34.2006554 39.44462454 48.57883751 10.42727898 44.33115876
33.38297565 46.70657644 30.20806361 45.35342073 38.62761763 39.74122031
48.16328556 47.08995401 39.50629817 42.88874373 47.62686203 37.07265334
39.31237238 17.27419595 45.50185006 37.19366452 37.15641128 29.38831544
37.09150271 35.49924936 43.25356164 40.03662925 45.68544861 24.5363516
43.58063428 47.34586063 38.90221522 16.67760066 37.63675126 16.13863568
47.81303885 37.62284986 27.31910162 34.25788066 39.24318948 47.20407679
47.67251114 38.86075834 43.95969168 47.67251114 38.85062693 45.66151705
48.140461 48.28503892 39.49759038 47.67650069 44.49495519 37.21648907
38.35541565 33.42589843 47.18125223 40.76914148 32.40254547 43.57732605
41.76501685 39.77456626 -3.27418165 16.49030496 32.61835101 43.91404257
43.20784736 43.1393737 39.60964151 48.45018525 45.5163075 48.140461
35.18587701 46.15562372 39.45834725 39.43686313 39.17604041 33.42589843
40.24031752 42.98983301 40.81571451 38.2111218 44.37272459 44.38483105
40.72274056 43.29914558 44.00534079 37.40077604 40.65705794 43.1393737
37.28496274 46.05900026 48.45018525 45.53002408 18.84875761 40.5273964
40.38471344 45.87643308 37.26213818 45.59468186 16.67760066 17.27419595
33.68017582 40.240546 44.32707548 40.37749333 48.62784146 40.96811202
16.88302166 47.08995401 48.45018525 37.76293425 47.67650069 40.97322343
39.77936502 33.75041847 11.74059428 46.20127283 47.62686203 37.4668188
45.69499421 39.4947561 13.71435754 40.94939096 38.56251247 43.43201127
11.64269665 45.68411731 31.25756567 16.54381018 27.75388699 32.20736942
29.29701722 25.61271379 40.47408544 16.81454799 37.29211359 41.37616352
39.37252063 44.53841746 19.18519108 42.99808126 39.48826873 41.02074531
40.88996351 42.38138352 38.65044219 42.7517964 36.4194728 44.14228812
35.49924936 36.43521578 41.98686854 36.21296265 37.37626096 40.57772796
45.78784702 47.81303885 32.77951617 47.71816025 32.82711681 42.64171747
36.9059464 40.71291211 43.1393737 39.28587025 39.95167917 37.64567441
39.69880433 37.62284986 39.57227108 48.140461 16.65793295 40.48228099
24.25797393 34.39077965 33.42589843 16.77876085 11.85960985 38.32479018
32.01493611 45.03685637 46.27336688 41.54565026 41.59129937 39.62731659
34.95507691 44.85425993 34.38576402 40.82893713 37.46924971 43.4715794
43.23686602 37.24238747 38.18829725 0.31844625 37.78191164 34.20912817
47.78663392 30.36195174 47.15947413 2.94880549 44.01159831 42.64080942
47.67650069 39.71861482 35.22214968 29.62978508 44.07381446 33.44872299

38.79245383 40.33184422 32.89559048 38.39152854 43.04890038 39.74143937
45.50213802 42.49550629 43.27632103 45.54739427 40.67316496 49.27333867
42.1079236 29.22854356 48.28503892 38.01150251 47.93312792 45.92208219
38.85656611 27.90606576 43.60345884 37.74736767 47.78663392 42.08509905
43.66686215 44.53030556 37.65222596 48.60166206 42.94369964 44.15124816
41.34492454 29.95311229 38.6994598 37.73971832 16.61228384 41.33061975
48.08948743 37.78916873 29.56131141 37.87946452 33.52337084 44.00534079
45.27456841 25.28084206 43.95969168 33.77147404 34.21979021 41.0541508
39.41297344 47.13560312 44.78238999 33.5902668 44.94555815 40.80897961
45.43337639 39.9253526 16.51312952 38.98757426 45.49105356 41.83817994
34.63553314 37.91426679 48.44189017 43.53787531 35.38247917 16.49030496
34.15990883 11.43291984 36.25680987 47.2497259 35.64806374 41.12458414
35.18975758 38.31811232 40.73714972 47.0225268 40.16646411 41.42181263
47.83228303 38.08623039 19.69649871 48.45018525 34.4529367 -0.85269248
46.86634832 47.2497259 43.23067192 33.18948806 26.34450597 33.40307388
37.21648907 46.15562372 45.91212368 47.34586063 48.140461 31.17902459
41.64668798 38.31195453 38.92286214 41.62077347 37.30962086 29.56988449
33.58519135 33.21001565 43.10631873 29.45678911 44.07381446 46.59757391
40.91278806 47.81303885 45.74219791 36.50212791 34.07776336 48.53654324
37.04194518 47.67650069 44.48465645 47.00457978 20.67708732 39.50171339
16.8601971 45.01083364 48.45018525 30.95648534 31.21306443 39.56855672
39.57988928 34.41711595 36.84575596 38.96851126 12.59294312 41.25857454
34.70868843 38.41577103 34.07776336 45.66151705 40.19466841 32.40531297
45.53670267 41.56633228 43.95969168 43.59206212 46.67361373 31.27032281
29.91230099 17.2970205 47.85510759 42.68082186 47.18125223 47.64968659
46.79688315]

Weights: [-2.60317734 -9.07040791]

Bias: 37.966553636840494

Assignment 5

Building Machine Learning Models with Scikit-learn

graphicx

Problem Statement 1: Classification on the Iris Dataset

- (a) Load the Iris dataset from Seaborn's dataset module into a Pandas DataFrame.
- (b) Split the dataset into training and test data.
- (c) Train and evaluate the following classification models:
 - i. K-Nearest Neighbors (KNN).
 - ii. Gaussian Naive Bayes.
 - iii. Decision Tree.
 - iv. Random Forest
 - v. Support Vector Machine (SVM)
- (d) For each model:
 - i. Print the labels predicted by the model on the test data and compare them with the actual labels.
 - ii. Form and display the confusion matrix using the test data.

Problem Statement 2: K-Means Clustering on the Wine Dataset

- (e) Load the Wine dataset from Scikit-learn's dataset module into a Pandas DataFrame.
- (f) Preprocess the data by selecting relevant features and handling any missing values if present.
- (g) Apply K-Means clustering to the dataset with the number of clusters equal to the number of unique wine classes in the dataset.
- (h) Print the cluster labels assigned to each data point by the K-Means algorithm.
- (i) Compare the cluster labels with the actual wine class labels by calculating the accuracy of the clustering.
- (j) Visualize the clusters using a scatter plot or pairplot with different colors representing different clusters.

Problem Statement 3: Linear Regression on the California Housing Dataset

- (k) Load the California Housing dataset from Scikit-learn's dataset module into a Pandas DataFrame.
- (l) Preprocess the data by selecting relevant features and handling any missing values if present.
- (m) Split the dataset into training and test data.
- (n) Train a Linear Regression model on the training data.
- (o) Evaluate the model by predicting housing prices on the test data and calculating performance metrics such as Mean Squared Error (MSE) and R-squared.
- (p) Print the coefficients of the model to understand the influence of each feature on the target variable.

1.1 Loading the Dataset

```
1 import seaborn as sns
2 import pandas as pd
3 iris = sns.load_dataset('iris')
4 df = pd.DataFrame(iris)
5 print(df.head())
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa


```
[ ]
```

1.2 Split the Dataset

```
1 from sklearn.model_selection import train_test_split
2 train_df, test_df = train_test_split(df, test_size=0.2)
3 print("train data \n",train_df.head())
4 print("test data \n",test_df.head())
```

train data

	sepal_length	sepal_width	petal_length	petal_width	species
108	6.7	2.5	5.8	1.8	virginica
0	5.1	3.5	1.4	0.2	setosa
46	5.1	3.8	1.6	0.2	setosa
105	7.6	3.0	6.6	2.1	virginica
87	6.3	2.3	4.4	1.3	versicolor

test data

	sepal_length	sepal_width	petal_length	petal_width	species
137	6.4	3.1	5.5	1.8	virginica
97	6.2	2.9	4.3	1.3	versicolor
13	4.3	3.0	1.1	0.1	setosa
3	4.6	3.1	1.5	0.2	setosa
93	5.0	2.3	3.3	1.0	versicolor

1.3 Load the Wine Dataset

```

1 X = df.drop('species', axis=1)
2 y = df['species']
3 import pandas as pd
4 from sklearn.metrics import confusion_matrix, accuracy_score
5 import seaborn as sns
6 import matplotlib.pyplot as plt
7 from sklearn.neighbors import KNeighborsClassifier
8 from sklearn.naive_bayes import GaussianNB
9 from sklearn.tree import DecisionTreeClassifier
10 from sklearn.ensemble import RandomForestClassifier
11 from sklearn.svm import SVC
12
13 def evaluate_model(model, X_test, y_test):
14     y_pred = model.predict(X_test)
15     print("Predicted Labels:")
16     print(y_pred)
17     print("Actual Labels:")
18     print(y_test)
19
20     cm = confusion_matrix(y_test, y_pred)
21     print(cm)
22     accuracy = accuracy_score(y_test, y_pred)
23     print(f"Accuracy: {accuracy}")
24
25 knn = KNeighborsClassifier(n_neighbors=5)
26 knn.fit(X_train, y_train)
27 print("K-Nearest Neighbors:")
28 evaluate_model(knn, X_test, y_test)
29
30 nb = GaussianNB()
31 nb.fit(X_train, y_train)
32 print("Gaussian Naive Bayes:")
33 evaluate_model(nb, X_test, y_test)
34
35 dt = DecisionTreeClassifier()
36 dt.fit(X_train, y_train)
37 print("Decision Tree:")
38 evaluate_model(dt, X_test, y_test)
39
40 rf = RandomForestClassifier()
41 rf.fit(X_train, y_train)
42 print("Random Forest:")
43 evaluate_model(rf, X_test, y_test)
44
45 svm = SVC()
46 svm.fit(X_train, y_train)
47 print("Support Vector Machine:")
48 evaluate_model(svm, X_test, y_test)

```

K-Nearest Neighbors:

Predicted Labels:

```
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']
```

Actual Labels:

73	versicolor
18	setosa
118	virginica
78	versicolor
76	versicolor
31	setosa
64	versicolor
141	virginica
68	versicolor
82	versicolor
110	virginica
12	setosa
36	setosa
9	setosa
19	setosa
56	versicolor
104	virginica
69	versicolor
55	versicolor
132	virginica
29	setosa
127	virginica
26	setosa
128	virginica
131	virginica
145	virginica
108	virginica
143	virginica
45	setosa
30	setosa

```

Name: species, dtype: object
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Accuracy: 1.0
Gaussian Naive Bayes:
Predicted Labels:
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']
Actual Labels:
73      versicolor
18         setosa
118      virginica
78      versicolor
76      versicolor
31         setosa
64      versicolor
141      virginica
68      versicolor
82      versicolor
110      virginica
12         setosa
36         setosa
9         setosa
19         setosa
56      versicolor
104      virginica
69      versicolor
55      versicolor
132      virginica
29         setosa
127      virginica
26         setosa
128      virginica
131      virginica

```

```

145     virginica
108     virginica
143     virginica
45       setosa
30       setosa
Name: species, dtype: object
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Accuracy: 1.0
Decision Tree:
Predicted Labels:
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']
Actual Labels:
73     versicolor
18       setosa
118     virginica
78     versicolor
76     versicolor
31       setosa
64     versicolor
141     virginica
68     versicolor
82     versicolor
110     virginica
12       setosa
36       setosa
9        setosa
19       setosa
56     versicolor
104     virginica
69     versicolor
55     versicolor
132     virginica

```

```

29      setosa
127     virginica
26      setosa
128     virginica
131     virginica
145     virginica
108     virginica
143     virginica
45      setosa
30      setosa
Name: species, dtype: object
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Accuracy: 1.0
Random Forest:
Predicted Labels:
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']
Actual Labels:
73     versicolor
18      setosa
118     virginica
78     versicolor
76     versicolor
31      setosa
64     versicolor
141     virginica
68     versicolor
82     versicolor
110     virginica
12      setosa
36      setosa
9       setosa
19      setosa

```

```

56      versicolor
104      virginica
69      versicolor
55      versicolor
132      virginica
29      setosa
127      virginica
26      setosa
128      virginica
131      virginica
145      virginica
108      virginica
143      virginica
45      setosa
30      setosa

```

Name: species, dtype: object

```

[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```

Accuracy: 1.0

Support Vector Machine:

Predicted Labels:

```

['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']

```

Actual Labels:

```

73      versicolor
18      setosa
118     virginica
78      versicolor
76      versicolor
31      setosa
64      versicolor
141     virginica
68      versicolor
82      versicolor

```

```

110    virginica
12      setosa
36      setosa
9      setosa
19      setosa
56    versicolor
104   virginica
69    versicolor
55    versicolor
132   virginica
29      setosa
127   virginica
26      setosa
128   virginica
131   virginica
145   virginica
108   virginica
143   virginica
45      setosa
30      setosa
Name: species, dtype: object
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Accuracy: 1.0

```

1.4 Preprocess the Data

```

1  from sklearn.datasets import load_wine
2  import pandas as pd
3
4  wine = load_wine()
5  df = pd.DataFrame(data=wine.data, columns=wine.feature_names)
6  df['target'] = wine.target
7  print(df.head())

```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	\
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	
3	14.37	1.95	2.50	16.8	113.0	3.85	

4	13.24	2.59	2.87	21.0	118.0	2.80
	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	\
0	3.06	0.28	2.29	5.64	1.04	
1	2.76	0.26	1.28	4.38	1.05	
2	3.24	0.30	2.81	5.68	1.03	
3	3.49	0.24	2.18	7.80	0.86	
4	2.69	0.39	1.82	4.32	1.04	
	od280/od315_of_diluted_wines	proline	target			
0	3.92	1065.0	0			
1	3.40	1050.0	0			
2	3.17	1185.0	0			
3	3.45	1480.0	0			
4	2.93	735.0	0			

1.5 Apply K-Means Clustering

```

1 df.isnull().sum()
2 df.dropna(inplace=True)
3 df.fillna(df.mean(), inplace=True)
4 from sklearn.feature_selection import SelectKBest, f_classif
5 selector = SelectKBest(f_classif, k=5)
6 X_new = selector.fit_transform(df.drop('target', axis=1), df['target'])
7 selected_features = df.drop('target', axis=1).columns[selector.get_support()]
8 df = df[selected_features]
9 df['target'] = wine.target

```

	0
alcohol	0
malic_acid	0
ash	0
alcalinity_of_ash	0
magnesium	0
total_phenols	0
flavanoids	0
nonflavanoid_phenols	0
proanthocyanins	0
color_intensity	0
hue	0
od280/od315_of_diluted_wines	0

```
proline                0
target                 0
dtype: int64
```

1.6 Print Cluster Labels

```
1 from sklearn.cluster import KMeans
2
3 n_clusters = len(df['target'].unique())
4
5 kmeans = KMeans(n_clusters=n_clusters, random_state=42)
6
7 kmeans.fit(df.drop('target', axis=1))
8
9
10 labels = kmeans.labels_
11
12 df['cluster'] = labels
13 print(df.head())
14 df.to_csv('clustered_wine_data.csv', index=False)
```

	alcohol	flavanoids	color_intensity	od280/od315_of_diluted_wines \
0	14.23	3.06	5.64	3.92
1	13.20	2.76	4.38	3.40
2	13.16	3.24	5.68	3.17
3	14.37	3.49	7.80	3.45
4	13.24	2.69	4.32	2.93

	proline	target	cluster
0	1065.0	0	1
1	1050.0	0	1
2	1185.0	0	1
3	1480.0	0	1
4	735.0	0	0

1.7 Compare Cluster Labels with Actual Labels

```
1 print(df['cluster'])
```

```
0      1
1      1
2      1
3      1
4      0
..
173    0
174    0
175    0
176    0
177    2
```

Name: cluster, Length: 178, dtype: int32

1.8 Compare Cluster Labels with Actual Labels

```
1 print(accuracy_score(df['target'], df['cluster']))
```

0.1853932584269663

1.9 Visualize the Clusters

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 sns.pairplot(df, hue='cluster', palette='viridis')
5 plt.show()
```

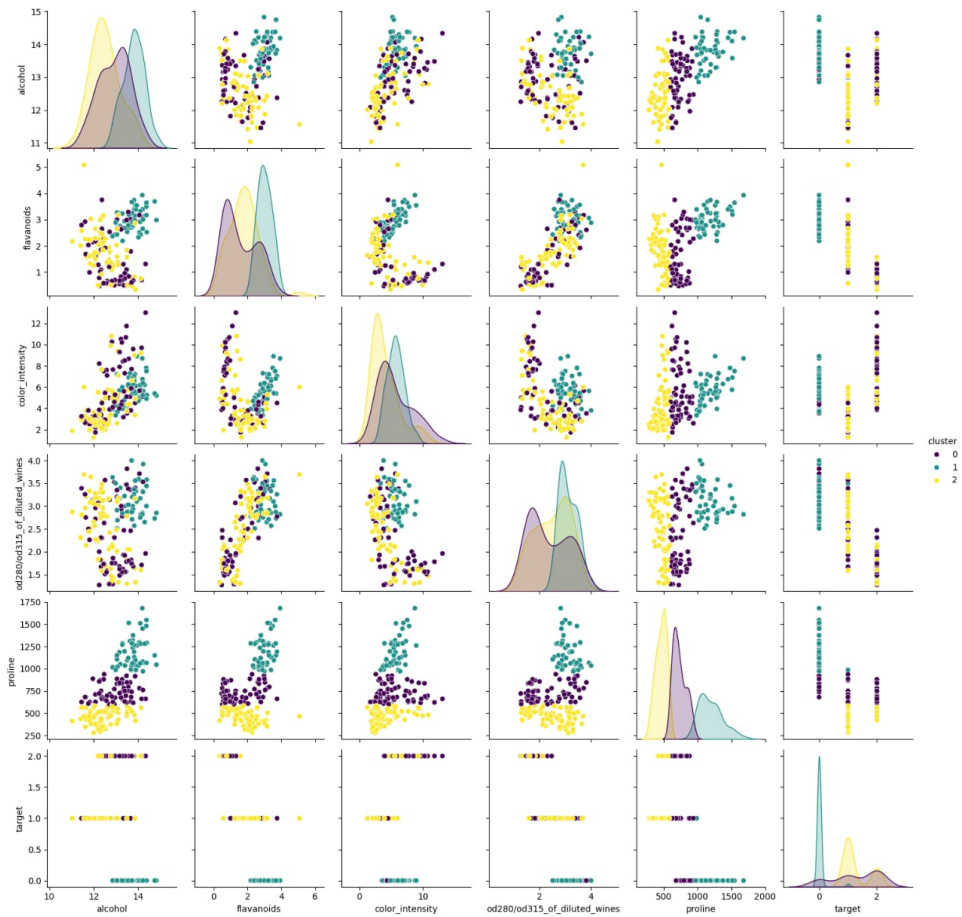


Figure 1.5:

Problem Statement 3: Linear Regression on the California Housing Dataset

1.10 Load the California Housing Dataset

```

1  from sklearn.datasets import fetch_california_housing
2  import pandas as pd
3  housing = fetch_california_housing()
4  df = pd.DataFrame(data=housing.data, columns=housing.feature_names)
5  df['target'] = housing.target
6  print(df.head())

```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	target
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

1.11 Preprocess the Data

```

1 df.isnull().sum()
2 df.fillna(df.mean(), inplace=True)
3 from sklearn.feature_selection import SelectKBest, f_regression
4 selector = SelectKBest(f_regression, k=5)
5 X_new = selector.fit_transform(df.drop('target', axis=1), df['target'])
6 selected_features = df.drop('target', axis=1).columns[selector.↵
    get_support()]

```

1.12 Split the Dataset

```

1 X_train, X_test, y_train, y_test = train_test_split(X_california, ↵
    y_california, test_size=0.3, random_state=42)

```

1.13 Train a Linear Regression Model

```

1 from sklearn.model_selection import train_test_split
2 X = df.drop('target', axis=1)
3 y = df['target']
4 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ↵
    random_s
5 from sklearn.linear_model import LinearRegression
6 model = LinearRegression()
7 model.fit(X_train, y_train)

```

```

LinearRegression
LinearRegression()

```

1.14 Evaluate the Model

```
1 from sklearn.metrics import mean_squared_error, r2_score
2 y_pred = model.predict(X_test)
3 mse = mean_squared_error(y_test, y_pred)
4 r2 = r2_score(y_test, y_pred)
5 print(f'Mean Squared Error: {mse}')
6 print(f'R-squared: {r2}')
```

Mean Squared Error: 0.6382565441555921

R-squared: 0.5129333248216971

1.15 Print the Coefficients

```
1 print(f'Coefficients: {model.coef_}')
2 print(df[['X3 distancetothenearrestMRTstation', 'decimal_scaled']].head())
```

Coefficients: [0.53520156 0.01618449 -0.20927062 1.050567 -0.03105515]

Assignment 6

Implementing an FCNN from Scratch using TensorFlow

Problem Statement:

19. Implement the following computations using TensorFlow:

- (a) Load the the MNIST dataset from tensorflow as x train, y train, x test and y test. The Modified National Institute of Standards and Technology (MNIST) dataset contains grayscale images of handwritten digits. The training set consists of 60,000 images and the test set contains 10,000 images. The label of each image is a digit between 0 and 9. Each image has a size of 28×28 , consisting of 784 pixel values, where each pixel value $[0, 255]$ with 0 corresponds to black, 255 to white, and values in between representing various shades of gray.
- (b) Compute X as the transpose of U.
- (c) Normalize the pixel values of X to $[0, 1]$ by dividing by 255.
- (d) Form a matrix Y of size m corresponding to the labels $[0, 9]$ of images by transposing y train.
- (e) Form a matrix V by reshaping the images in x test to be 1D arrays of 784 (28×28) pixel values (Flatten the images).
- (f) Compute Xtest as the transpose of V .
- (g) Normalize the pixel values of Xtest to $[0, 1]$ by dividing by 255.
- (h) Form a matrix Y test of size m corresponding to the labels $[0, 9]$ of images by transposing y test.
- (i) Select an image from X and display it. Also, display the corresponding label from Y.
- (j) Form a matrix Y of size m corresponding to the labels $[0, 9]$ of images by transposing y train.
- (k) Set the hyper parameters: $p = 10$, the no. of neurons in hidden layer, $q = 10$, the no. of neurons in output layer (corresponding 10 labels in one-hot encoding format), learning rate $= 0.01$ and the number of training epochs (iterations over the dataset) as 1000.
- (l) Create a matrix W1 of shape (p, n) and initialize it as $W1 = N(0, 1) \times q$ $1/n$, where $N(0, 1)$ represents a matrix of random values drawn from a normal distribution with mean 0 and standard deviation 1.

- (m) Initialize the vector B_1 of shape $(p, 1)$ to zeros.
- (n) Initialize the matrix W_2 of shape (q, p) as $W_2 = \mathcal{N}(0, 1) \times \frac{q}{p}$.
- (o) Initialize the vector B_2 of shape $(q, 1)$ to zeros.
- (p) Perform the following forward propagation and backpropagation computations iteratively (No. of epochs=1000):
 - i. $Z_1 = W_1 \cdot X + B_1$ (Matrix Multiplication)
 - ii. $A_1 = \text{ReLU}(Z_1)$ where $\text{ReLU}(x)$ is a function that returns 0 for negative values and the input value itself otherwise.
 - iii. $Z_2 = W_2 \cdot A_1 + B_2$
 - iv. $A_2 = \text{softmax}(Z_2)$, where $\text{softmax}(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}$.
 - v. Get the predicted labels from the output of A_2 (index of the maximum value).
 - vi. Find the accuracy of the predictions by comparing them to the true labels Y and print the progress in every 100 epochs.
 - vii. Compute the cross-entropy loss using TensorFlow's `tf.nn.softmax cross entropy` with logits function.
 - viii. $dZ_2 = A_2 - Y$ where one hot Y is the one-hot encoded form of Y .
 - ix. $dA_2 = W_2^T \cdot dZ_2$
 - x. $dW_2 = \frac{1}{m} \cdot dZ_2 \cdot A_1^T$
 - xi. $dB_2 = \frac{1}{m} \sum dZ_2$ (sum along the columns)
 - xii. $dZ_1 = dA_2 \circ \text{ReLU deriv}(Z_1)$ where $\text{ReLU deriv}(x)$ returns 1 for positive values and 0 otherwise, and $\circ$ indicates element-wise multiplication.
 - xiii. $dA_1 = W_1^T \cdot dZ_1$
 - xiv. $dB_1 = \frac{1}{m} \sum dZ_1$ (sum along the columns)
 - xv. $dW_1 = \frac{1}{m} \cdot dZ_1 \cdot X^T$
 - xvi. (p) Update and print W_1 , B_1 , W_2 , and B_2 for $\alpha = 0.01$:
 - i. $W_1 = W_1 - \alpha \cdot dW_1$
 - ii. $B_1 = B_1 - \alpha \cdot dB_1$
 - iii. $W_2 = W_2 - \alpha \cdot dW_2$
 - iv. $B_2 = B_2 - \alpha \cdot dB_2$
- (q) . Use tensorflow `GradientTape()` to automatically calculate the gradients from steps (h) to (o) and redo the training steps.
- (r) Select one test image from X_{test} , display it, reshape it to $n \times 1$, perform forward propagation computations and predict the label. Check whether the prediction is correct.

- (s) Use the entire X_{test} and perform the forward propagation computations and predict the accuracy of the model.

1.1 Load and Preprocess Data

```
1 import numpy as np
2 from matplotlib import pyplot as plt
3 import tensorflow as tf
4 (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
5 U = tf.reshape(x_train, (x_train.shape[0], 784))
6 X = tf.transpose(U)
7 X = tf.cast(X, tf.float32) / 255.0
8 Y = tf.transpose(tf.convert_to_tensor(y_train))
9 n, m = X.shape
10 V = tf.reshape(x_test, (x_test.shape[0], 784))
11 Xtest = tf.transpose(V)
12 Xtest = tf.cast(Xtest, tf.float32) / 255.0
13 Ytest = tf.transpose(tf.convert_to_tensor(y_test))
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/11490434/11490434> 0s 0us/step

1.2 Display an Image

```
1 ind = 0
2 image = X[:, ind]
3 image = tf.reshape(image, (28, 28)) * 255
4 image_np = image.numpy()
5 plt.imshow(image_np, cmap='gray', interpolation='nearest')
6 print(f"Label: {Y[ind].numpy()}")
7 plt.show()
```

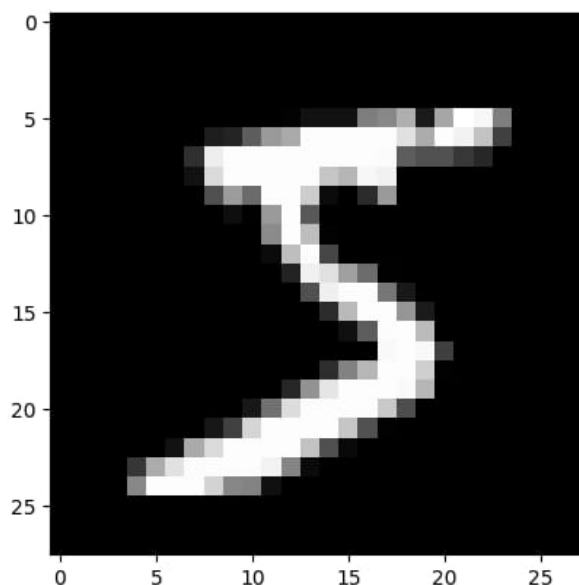


Figure 1.6: Image Visualization

1.3 Neural Network Parameters

```
1 p = 10
2 c = 10
3 alpha = 0.01
4 epoch = 1000
5 W1 = tf.Variable(tf.random.normal((p, n), stddev=tf.math.sqrt(1. / n)))
6 b1 = tf.Variable(tf.zeros((p, 1)))
7 W2 = tf.Variable(tf.random.normal((c, p), stddev=tf.math.sqrt(1. / p)))
8 b2 = tf.Variable(tf.zeros((c, 1)))
```

1.4 Activation Functions and Encoding

```
1 def ReLU(Z):
2     return tf.maximum(Z, 0)
3 def ReLU_deriv(Z):
4     return tf.cast(Z > 0, dtype=tf.float32)
5 def softmax(Z):
6     exp_Z = tf.exp(Z - tf.reduce_max(Z))
7     return exp_Z / tf.reduce_sum(exp_Z, axis=0, keepdims=True)
8 def one_hot(Y):
9     return tf.one_hot(Y, depth=tf.reduce_max(Y) + 1)
```

1.5 Training Loop

```
1 for i in range(epoch):
2     Z1 = tf.matmul(W1, X) + b1
3     A1 = ReLU(Z1)
4     Z2 = tf.matmul(W2, A1) + b2
5     A2 = softmax(Z2)
6     one_hot_Y = one_hot(tf.cast(Y, dtype=tf.int32))
7     one_hot_Y = tf.transpose(one_hot_Y)
8     if i % 100 == 0:
9         yhat = tf.argmax(A2, axis=0)
10    accuracy=tf.reduce_mean(tf.cast(yhat==tf.cast(Y,dtype=tf.int64),dtype=tf.float32)).numpy() * 100
11    print(f"Epoch: {i}, Accuracy: {accuracy:.2f}%")
```

```

1 dZ2 = A2 - one_hot_Y
2 dW2 = 1 / m * tf.matmul(dZ2, tf.transpose(A1))
3 db2 = 1 / m * tf.reduce_sum(dZ2, axis=1, keepdims=True)
4 dA2 = tf.matmul(tf.transpose(W2), dZ2)
5 dZ1 = dA2 * ReLU_deriv(Z1)
6 dW1 = 1 / m * tf.matmul(dZ1, tf.transpose(X))
7 db1 = 1 / m * tf.reduce_sum(dZ1, axis=1, keepdims=True)
8 W2.assign(W2 - alpha * dW2)
9 b2.assign(b2 - alpha * db2)
10 W1.assign(W1 - alpha * dW1)
11 b1.assign(b1 - alpha * db1)

```

```

Epoch: 0, Accuracy: 7.20%
Epoch: 100, Accuracy: 31.82%
Epoch: 200, Accuracy: 47.35%
Epoch: 300, Accuracy: 66.75%
Epoch: 400, Accuracy: 74.39%
Epoch: 500, Accuracy: 78.53%
Epoch: 600, Accuracy: 80.75%
Epoch: 700, Accuracy: 82.20%
Epoch: 800, Accuracy: 83.30%
Epoch: 900, Accuracy: 84.22%

```

1.6 Make Predictions

```

1 def show_prediction(index, W1, b1, W2, b2):
2     testimage = Xtest[:, index]
3     testimage = tf.reshape(testimage, (784, 1))
4     Z1 = tf.matmul(W1, testimage) + b1
5     A1 = ReLU(Z1)
6     Z2 = tf.matmul(W2, A1) + b2
7     A2 = softmax(Z2)
8     yhat = tf.argmax(A2, axis=0).numpy()
9     label = Ytest[index].numpy()
10    testimage = tf.reshape(testimage, (28, 28)) * 255
11    plt.gray()
12    plt.imshow(testimage, interpolation='nearest')
13    plt.show()
14    print("Prediction:", yhat)
15    print("Actual Label:", label)
16    index=39

```

```
17 show_prediction(index, W1, b1, W2, b2)
```

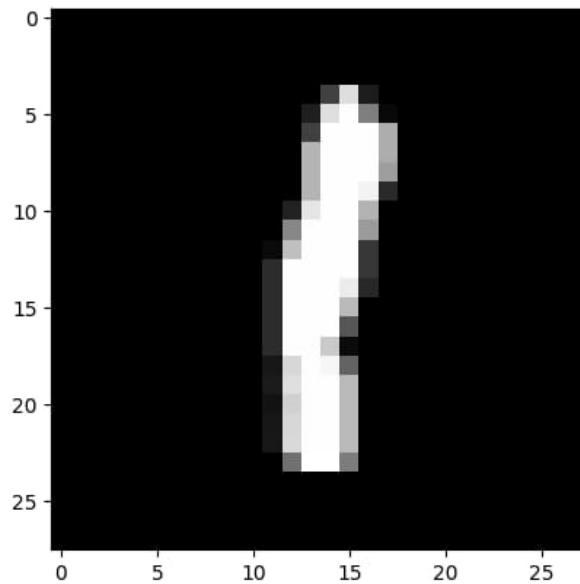


Figure 1.7: Image Visualization

1.7 Evaluate Model

```
1 def test(Xtest, Ytest, W1, b1, W2, b2):
2     Z1 = tf.matmul(W1, Xtest) + b1
3     A1 = ReLU(Z1)
4     Z2 = tf.matmul(W2, A1) + b2
5     A2 = softmax(Z2)
6     yhat = tf.argmax(A2, axis=0).numpy()
7     accu = tf.reduce_mean(tf.cast(tf.equal(yhat, Ytest), dtype=tf.float32)).↵
           numpy() * 100
8     print("Predictions:", yhat)
9     print("Actual Labels:", Ytest.numpy())
10    print("Accuracy:", accu, "%")
11    test(Xtest, Ytest, W1, b1, W2, b2)
```

```
Predictions: [7 2 1 ... 4 8 6]
Actual Labels: [7 2 1 ... 4 5 6]
Accuracy: 85.6000006198883 %
```

1.8 Error, Precision, Recall, f1 score

```
1 from sklearn.metrics import confusion_matrix, precision_score, ←
   recall_score, f1_score
2 def test_and_evaluate(Xtest, Ytest, W1, b1, W2, b2):
3     Z1 = tf.matmul(W1, Xtest) + b1
4     A1 = ReLU(Z1)
5     Z2 = tf.matmul(W2, A1) + b2
6     A2 = softmax(Z2)
7     yhat = tf.argmax(A2, axis=0).numpy()
8     y_true = Ytest.numpy()
9     cm = confusion_matrix(y_true, yhat)
10    error_rate = 1 - np.trace(cm) / np.sum(cm)
11    precision = precision_score(y_true, yhat, average='weighted')
12    recall = recall_score(y_true, yhat, average='weighted')
13    f1 = f1_score(y_true, yhat, average='weighted')
14    print("Confusion Matrix:\n", cm)
15    print("Error Rate:", error_rate)
16    print("Precision:", precision)
17    print("Recall:", recall)
18    print("F1 Score:", f1)
19    test_and_evaluate(Xtest, Ytest, W1, b1, W2, b2)
```

Confusion Matrix:

```
[[ 935  0  2  6  1  8 17  2  9  0]
 [  0 1103  2  5  1  0  4  1 19  0]
 [ 18 23 849 50 20  1 29 19 19  4]
 [ 10  6 35 839  2 45  7 10 38 18]
 [  1  9  6  0 831  2 13  0 12 108]
 [ 24 15  7 95 30 634 24 15 32 16]
 [ 24  8  7  0 13 20 877  0  9  0]
 [  8 41 28  3  9  1  1 881 12 44]
 [  5 29  8 38 23 25 14 19 795 18]
 [ 10 11  9  7 94 20  1 26 15 816]]
```

Error Rate: 0.14400000000000002

Precision: 0.8560334612260678

Recall: 0.856

F1 Score: 0.8549597853086216